

Meta-programming is the art of writing programs that produce or manipulate other programs. This allows programmers to automate error-prone or repetitive tasks, and exploit domain-specific knowledge to customize the generated code.

Unfortunately, writing safe meta-programs remains very challenging and sometimes frustrating, as traditionally errors in the generated code are only detected when running it, but not at the time when code is generated. Moreover, while there is ad-hoc support for meta-programming in type theories, there is no proper foundation for it.

This talk revisits Cocon, a framework for certified meta-programming.

Cocon, a Martin-Loef dependent type theory for defining logics and proofs that allows us to represent domain-specific languages (DSL) within the logical framework LF and in addition write

recursive programs and proofs about those DSLs.

In particular, we can embed into LF STLC or System F, etc. and then write programs about those encodings using Cocon itself. This means Cocon is a target for compiling meta-programming systems --

from compiling meta-programming with STLC or System F to compiling meta-programming in CoC. We then motivate the design of DeLAM, a dependent type theory that support meta-programming by distinguishing

between the code that is being generated, the type of that code, and the program that is actually generating the code.

I will also highlight more recent research directions that aim to support meta-programming and intensional code analysis.